

1. Create file: procinfo.h

add this code:

```
#ifndef PROCINFO_H
#define PROCINFO_H
#include "types.h"

struct procinfo {
    int pid;
    char name[16];
    char state[16];
    uint sz;
};

#endif
```

2. Modify: proc.c

add this code:

```
#include "procinfo.h"
int
get_procinfo_kernel(int pid, struct procinfo *info)
{
    struct proc *p;

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->pid == pid && p->state != UNUSED)
            info->pid = p->pid;
            safestrcpy(info->name, p->name, sizeof(info->name));

            char *st = "UNKNOWN";
            switch(p->state)
            {
                case UNUSED: st = "UNUSED"; break;
                case EMBRYO: st = "EMBRYO"; break;
                case SLEEPING: st = "SLEEPING"; break;
                case RUNNABLE: st = "RUNNABLE"; break;
                case RUNNING: st = "RUNNING"; break;
                case ZOMBIE: st = "ZOMBIE"; break;
            }

            safestrcpy(info->state, st, sizeof(info->state));
            info->sz = p->sz;

            release(&ptable.lock);
            return 0;
}
```

3. Modify: defs.h

add this line

```
#include "procinfo.h"
int get_procinfo_kernel(int pid, struct procinfo *info);
```

4. Modify syscall.h

add this line

```
#define SYS_get_procinfo 29
```

5. Modify sysproc.c

add this code line:

```
#include "procinfo.h"
int
```

```

sys`get`proc`info(void)
-
int pid;
char *uaddr;          // pointer in user space
struct proc`info info;

if(argint(0, &pid) != 0)
    return -1;

if(argptr(1, &uaddr, sizeof(info)) != 0)
    return -1;

if(get`proc`info`kernel(pid, &info) != 0)
    return -1;

if(copyout(myproc()->pgdir, (uint)uaddr, (char*)&info, sizeof(info)) != 0)
    return -1;

return 0;

```

6. Modify syscall.c

add this code line:

```

extern int sys`get`proc`info(void);

[SYS`get`proc`info] sys`get`proc`info,

```

7. Modify usys.S

add this line:

```

SYSCALL(get`proc`info)

```

8. Modify user.h

add this code:

```

struct proc`info {
    int    pid;
    char   name[16];
    char   state[16];
    uint   sz;
};

int get`proc`info(int pid, struct proc`info *info);

```

9. create pinfo.c

```

#include "types.h"
#include "user.h"

int
main(int argc, char *argv[])
-
    if(argc != 2)-
        printf(2, "usage: pinfo %pid\n");
        exit();

    int pid = atoi(argv[1]);
    struct proc`info info;

    if(get`proc`info(pid, &info) != 0)-

```

```

    printf(1, "Invalid PID or not found\n");
    exit();

    printf(1, "PID: %d\n", info.pid);
    printf(1, "Name: %s\n", info.name);
    printf(1, "State: %s\n", info.state);
    printf(1, "Size: %d\n", info.sz);

    exit();

```

10. create testproc.c

```

#include "types.h"
#include "user.h"

int
main(void)
{
    int i;
    int num_children = 5;

    for(i = 0; i < num_children; i++) {
        int pid = fork();
        if(pid < 0) {
            printf(1, "Fork failed\n");
            exit();
        }

        if(pid == 0) {
            printf(1, "Child process %d started with PID %d\n", i+1, getpid());
            while(1); // loop forever
        }
    }
    exit();
}

```

11. Edit MAKEFILE

add this in UPROGS:

```

testproc "
pinfo

```

ScreenShots:

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Downloads/xv6-public
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Downloads/xv6-public$ make qemu
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
12340920$testproc
Child process 1 started with PID 4
Child process 2 started with PID 5
Child process 3 started with PID 6
Child process 4 started with PID 7
Child process 5 started with PID 8
12340920$pinf 4
PID: 4
Name: testproc
State: RUNNABLE
Size: 12288
12340920$pinf 6
PID: 6
Name: testproc
State: RUNNABLE
Size: 12288
12340920$
```

```
QEMU

Machine View

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
12340920$testproc
Child process 1 started with PID 4
Child process 2 started with PID 5
Child process 3 started with PID 6
Child process 4 started with PID 7
Child process 5 started with PID 8
12340920$pinf 4
PID: 4
Name: testproc
State: RUNNABLE
Size: 12288
12340920$pinf 6
PID: 6
Name: testproc
State: RUNNABLE
Size: 12288
12340920$_
```